key.c | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 按键模块：实现PA5/PA6/PA7低有效按键扫描、20ms消抖与按下事件。 */
#include "key.h"
#include "hmi_port.h"
#define KEY_COUNT 3U
#define DEBOUNCE_MS 20U
static uint8_t candidate[KEY_COUNT], stable[KEY_COUNT];
static volatile uint8_t pending;
static uint32_t changed_at[KEY_COUNT];
/**
 * 功能：初始化按键 GPIO 并复位消抖状态
 * 参数：无
 * 返回：无
 * 注意：启动时调用；按下状态从下一次扫描开始计时
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Init。
 */
void Key_Init(void)
{
    uint8_t i;
    HMI_PortInit(); pending=0;
    for (i=0; i<KEY_COUNT; ++i) {
        candidate[i]=0; stable[i]=0; changed_at[i]=0;
    }
}
/**
 * 功能：非阻塞扫描三键，连续稳定20ms后更新状态
 * 参数：now_ms：单调递增毫秒节拍，允许32位回绕
 * 返回：无
 * 注意：每1~5ms调用；可在节拍中断调用；仅允许一个扫描调用者
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Scan。
 */
void Key_Scan(uint32_t now_ms)
{
    uint8_t i, raw=HMI_KeyRead();
    for (i=0; i<KEY_COUNT; ++i) {
        uint8_t pressed=(uint8_t)((raw>>i)&1U);
        if (pressed!=candidate[i]) {
            candidate[i]=pressed; changed_at[i]=now_ms;
        } else if (stable[i]!=pressed &&
                   (uint32_t)(now_ms-changed_at[i])>=DEBOUNCE_MS) {
            stable[i]=pressed;
            if (pressed) pending|=(uint8_t)(1U<<i);
        }
    }
}
/**
 * 功能：取出一个消抖后的按下键值
 * 参数：无
 * 返回：KEY_NONE或三个功能键值
 * 注意：主循环调用；同一键未消费的多次事件合并；同时按下按键号顺序返回
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_GetValue。
 */
key_value_t Key_GetValue(void)
{
    uint8_t i; uint32_t mask=HMI_EnterCritical();
    for (i=0; i<KEY_COUNT; ++i) {
        if (pending&(1U<<i)) {
            pending&=(uint8_t)~(1U<<i);
            HMI_ExitCritical(mask);
```

```
            return (key_value_t)(i+1U);
        }
    }
    HMI_ExitCritical(mask); return KEY_NONE;
}
```